

Lab 1: Scanning

Follow [Chapter 4: Scanning](#) of *Crafting Interpreters*. Design your own language, using Lox and the chapter's implementation as references. Choose your language's name, keywords, operators, and lexical rules, and explain your design choices and differences from Lox in your report. This lab covers scanning only; parsing and execution are not required.

Python is the preferred implementation language, but you may choose another language if you prefer.

Tasks

You will submit both your **code** and a **report** for this lab. Partial credit will be awarded for all tasks.

Report quality and submission correctness together account for at most **10% of the total score**, included within the task weights below.

- **Interpreter entry point (20%):**

Create a basic entry point (main class and function) of your language interpreter. It should support:

1. Running from a source file.
2. Interactive mode, where the user can type code directly into the terminal.
3. Connect the entry point to the scanner so that it takes source text as input and prints a list of tokens.

- **Scanner implementation (40%):**

Implement all token types defined in your language's lexical grammar, using Lox's scanner as a reference. Include identifiers, keywords, numeric and string literals, operators, punctuation, and an end-of-file (EOF) token. Define how whitespace and comments are handled. Create a **Token** class or equivalent structure storing:

- token type
- lexeme (raw string input)
- literal value (internal representation in your interpreter)
- line information for lexical errors

Use this scanner in the main file and print out the scanned tokens. The output for each token should contain the token type, lexeme, and the literal value.

- **Error Handling (20%):** Your scanner must detect and report lexical errors, including unterminated strings and unexpected characters, as well as errors introduced by your own lexical rules. Each diagnostic must identify the problem and its line number. Interactive mode must remain usable after an error.
- **Lexical grammar design (20%):** Write regular expressions for your language's **number literals**, **string literals**, and **identifiers** in your report.

Testing and Report

Write your own test cases covering **every token type and all edge cases for your language's lexical rules**, including invalid input and every supported lexical error. Test both source-file and interactive modes, including recovery after errors.

Place your report under **report/** in your repository. Your report must include:

- Your language's name, regular expressions for number literals, string literals, and identifiers, and design choices relative to Lox.
- Dependency/setup instructions and commands to run the scanner in both source-file and interactive modes and reproduce the tests.
- For each test case: its purpose, source input, expected tokens or errors, actual scanner output, and whether the result matches expectations. Output may be included as text or readable screenshots.
- Any known limitations or failing tests.

Commit the test inputs and any test scripts under **test/lab1/** so that your results can be reproduced.

Optional Extra Credit

You may apply for extra credit if you believe you put extra effort into a feature beyond the basic scanner requirements. Examples include nested block comments, scientific number literals, string escape sequences, or enhanced error diagnostics. In your report under **report/**, identify the feature, explain the additional effort and its value, and provide test cases and scanner output demonstrating it. Extra credit is subject to instructor evaluation and is capped at **10% of the lab's base grade (10 percentage points)** across all extra-credit features.

Submission Instructions

Submit a **link to the exact commit in your GitHub repository** on Blackboard, using the format **<https://github.com/<owner>/<repo>/commit/<commit-hash>>**.

The submitted commit must include:

- All implementation files in **src/**.
- All test inputs and test scripts for this lab in **test/lab1/**.
- The Lab 1 report in **report/**, including setup/run instructions, test results, and any extra-credit request.

Use the following repository structure (the report filename below is an example):

```
repository/
├── src/           # Implementation files
├── test/
│   └── lab1/     # Lab 1 test inputs and test scripts
├── report/
│   └── lab1.md   # Lab 1 report, including setup and run
                  instructions
```

Ensure that the instructor can access the repository and reproduce your results from the submitted commit.

Late Penalty

You should submit your lab on time. Being late for one lab could affect your time to complete subsequent labs. All labs are due at 11:59 pm (Eastern) of the specified due date, and there is **a 10% penalty each day for up to 50%**. After that, you get zero.

AI Policy

Using AI help for the labs are ALLOWED but discouraged. But you must quote the code from AI help.

Quote it like this:

```
// --- start AI code ---
```

```
// --- end AI code ---
```

Using AI generated code without quoting is considered academic misconduct and will result in 0 for the lab. Mistakes in the code from AI help will results in doubled point deduction according to the rubrics.

Reuse of any code of previous labs are ALLOWED and encouraged.